

# Software Design Document

## SCHEDULIX SYSTEM — VERSION 2.0

---

### 1. Introduction and General Architectural Approach

---

The objective of Version 2.0 is to add a Graphical User Interface (GUI) that provides visual functionality and enhanced usability for displaying schedules and exam planning.

Given the time constraints and the size of the team, the system is designed using a **Layered Architecture** approach. This approach allows us to reuse the core code from Version 1.0 (such as the course filtering mechanism and the schedule generation algorithm) without any modifications. The new UI layer will interface with the existing `ScheduleLixApp` class, which serves as the entry point to the business logic.

### 2. Selected Design Patterns

---

#### 2.1 MVP Pattern for Interface Planning (Instead of MVC)

**Where it is used:** Across all system screens, including the input screens and the calendar output screen.

**Why we chose it:** Unlike the MVC pattern—where the Controller and sometimes the View are intertwined with the logic—the **MVP (Model-View-Presenter)** pattern keeps the **View** completely passive. The **Presenter** receives events from the UI (such as clicking a data loading button), queries the Model, and returns simple display instructions back to the View.

**Professional Justification:** This project requires comprehensive unit testing and a rigorous code review process to prevent bugs. The MVP pattern allows us to write unit tests for the Presenter without needing to run a heavy GUI environment.

**Actual Implementation (Transitioning from Version 1.0 to MVP Architecture):** To implement this pattern efficiently within the existing timeframe, we will divide the work into the following components:

- **Model:** We will use the existing code from Version 1.0 without modification. The existing `SchedulixApp` class and scheduling components will serve as our data model and business logic.
- **View:** We will create new classes responsible solely for the visual user interface (buttons, checkboxes/combo boxes, calendar view). These classes will not perform any calculations or file reading; they will only forward click events.
- **Presenter:** We will create new classes (for example, `InputScreenPresenter`) that connect the View to the Model. The Presenter will hold an instance of the View and an instance of `SchedulixApp`. When a user clicks the "Show Schedule" button, the View will capture the request, the Presenter will call the appropriate function in `SchedulixApp`, and then update the View with the returned result.

## 2.2 State Management: Dependency Injection Instead of Singleton

**Where it is used:** Managing loaded data (courses, curriculums/study programs, dates).

**Why we chose it:** The requirements state that internal data caching must be supported to avoid reloading unmodified files and to maintain fast response times. The initial instinct might be to use a Singleton to store data in global memory. However, Singleton is an anti-pattern in the context of unit testing, as it preserves state between tests.

**Professional Justification:** We will implement a **Cache Manager** configured as a standard object. This object will be instantiated once and passed to the various screens as a parameter using the **Dependency Injection** method.

### Cache Manager Implementation via Serialization:

- **How we will implement it:** We will use Python's built-in serialization mechanism (the `pickle` library) to save the state of objects from RAM directly into a hidden internal file (e.g., `internal_data.pkl`).
- **Why we chose this implementation:**
  - **State Persistence Between Executions:** Data in RAM is erased when the software closes. Writing the objects using `pickle` allows the system to reload them instantly upon the next launch, eliminating the need to reload and re-parse the original CSV files.
  - **Flexible Support for Updates:** When the user chooses to update data (either overriding or incrementally adding), the Cache Manager updates the object in memory and immediately overwrites the internal file with the latest copy. This avoids expensive I/O operations and keeps the system responsive.

## 2.3 Command Pattern for Date Exceptions Management

**Where it is used:** In the UI screen displaying the calendar, where the user can select a specific day to exclude it or restore it to an active status.

**Why we chose it:** Clicking a date triggers a logical update in the date manager ( `ExamDateHandler` ). Wrapping each click action inside a **Command** object (which holds the date and the required action) allows for flexible execution and completely decouples the button click event from the list-modification logic.

## 2.4 Extending the Factory Pattern

**Where it is used:** Creating dynamic UI screen components.

**Why we chose it:** In Version 1.0, a Factory mechanism was already implemented to dynamically generate the various file readers ( `FileReaderFactory` ). We will extend this concept and create a `ViewFactory` for the user interface.

**Professional Justification:** The output screen is required to dynamically display one schedule out of many and allow pagination. Using a Factory to create calendar items and courses centralizes the UI component creation logic into a single place.

## 3. Interface Technology and Performance

---

### 3.1 Graphical Library Selection (customTkinter)

To implement the graphical interface, we chose to use the `customTkinter` library. This library provides a modern, lightweight wrapper around the built-in `Tkinter` , offering clean visual support with rounded corners. Unlike alternative libraries like `PyQt` —which have a very steep learning curve and high complexity regarding event management—this choice allows us to achieve a modern result in minimal time. Furthermore, the library's simplicity integrates perfectly with the MVP architecture we designed, as it does not enforce its own logical frameworks on the View, leaving us time to focus on unit testing and business logic.

### 3.2 Performance Management and Preventing UI Freezing

The requirements state that any delay or "freezing" longer than one second constitutes a lack of responsiveness. Generating scheduling options is a computationally heavy operation. If we run the algorithm directly on the interface's Main Event Loop, the screen will completely lock up and freeze until the calculation completes.

To solve this and maintain a responsive UI, we implemented asynchronous work using **Multi-threading**. Upon clicking "Schedule", the Presenter immediately switches the screen to a loading state while simultaneously launching the scheduling algorithm to run in the background on a separate **Worker Thread**.

Moreover, to prevent the calculation itself from taking upwards of 30 seconds, the algorithm utilizes a **Lazy Evaluation** technique. Instead of generating millions of options upfront to store in memory, it generates and returns only the first batch of schedules. The rest of the calculation occurs dynamically and on-demand, only when the user pages through to the subsequent schedules.

## 4. User Interface Design (UX/UI) and Workflow

---

In accordance with system requirements, an interactive user interface was designed to separate the data input phase from the results display phase.

### 4.1 UX/UI Design Concept: Multi-Step Workflow

To prevent visual overload and confusion for the user (such as administrative staff who may not necessarily have a technical background), the interface will not display all features on a single, cluttered screen. Instead, the data entry process will be designed as a **Multi-Step Workflow** based on sequential screens. Each screen will focus on a single task (loading data, selecting programs, managing dates), with transitions between them handled intuitively via "Next/Previous" buttons.

### 4.2 Main Menus and Screens

The system will feature several core views managed by the Presenter:

- **Input Screen - Phase A (Data Loading):** Features two primary action buttons: "Load Courses File" and "Load Dates File". The system will display a visual indicator (Green/Red) representing the status of data loading into the cache.
- **Input Screen - Phase B (Program Selection):** Displays a list ( **Listbox** ) of all study programs. Selecting a program opens a side view showing its associated mandatory and elective courses for verification purposes.
- **Input Screen - Phase C (Date Management):** Displays a calendar view. Clicking a date changes its color and toggles its status between "Excluded" and "Active".
- **Output Screen (Schedule View):** Displays the generated schedule on top of a calendar. The screen includes a bottom toolbar for pagination (Next/Previous), status text ("Schedule X out of Y"), and a "Save Schedule" button to export the result to a text file.

### 4.3 User Flow

The workflow reflects the user's natural progression through the system:

1. **Data Loading:** The user opens the application, loads the data files, and proceeds to the next screen.
2. **Program Selection:** The user selects the relevant study programs and proceeds to the next screen.
3. **Date Management:** The user excludes specific dates from the exam schedule as needed.
4. **Running the Algorithm:** The user clicks the "Schedule" button. The Presenter gathers the data, runs the core logic, and awaits the response.
5. **Viewing and Saving:** The system automatically navigates to the output screen, where the user browses the results, identifies the best schedule, and saves it.